

Small Language Models for Marine Hydrodynamics Problem Solving using RLPT

*Thesis to be submitted in the partial fulfillment of the requirements for
the degree*

MASTER OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE

by

Raghuveer Patil
(21NA3AI42)

Under the guidance of

Prof. Prabhat Kumar Mishra

Department of Artificial Intelligence
Indian Institute of Technology, Kharagpur
24th April, 2026

Department of Artificial Intelligence
Indian Institute of Technology,
Kharagpur
India – 721302

CERTIFICATE

This is to certify that we have examined the thesis entitled **Small Language Models for Marine Hydrodynamics Problem Solving using RLPT**, submitted by **Raghuveer Patil** (Roll Number: 21NA3AI42) a postgraduate student of Department of Artificial Intelligence in partial fulfillment for the award of degree of Master of Technology. We hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in partial fulfillment for the Post Graduate Degree for which it has been submitted. The thesis has fulfilled all the requirements as per the regulations of the Institute and has reached the standard needed for submission.

Supervisor

Department of Artificial Intelligence
Indian Institute of Technology, Kharagpur

Place: Kharagpur

Date:

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Prof. Prabhat Kumar Mishra, who introduced me to the exciting intersection of Reinforcement Learning and domain-specific language modelling. His profound expertise and insightful guidance have been instrumental in the progress and success of my research. I am also indebted to the faculty and staff of the Department of Artificial Intelligence, whose assistance has been invaluable throughout my academic journey. Their willingness to share knowledge and offer advice has greatly enriched my learning experience. Special thanks are due to my peers and colleagues who have provided me with support and collaboration. Their perspectives and feedback have been a source of encouragement and growth. Finally, and most importantly, my heartfelt thanks go to my family. Their unwavering support, endless patience, and belief in my abilities have been my strength and perseverance. This accomplishment is as much theirs as it is mine.

Abstract

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in general-purpose reasoning, yet these models frequently underperform on specialized engineering tasks requiring structured numerical reasoning and domain-specific knowledge. This thesis investigates the feasibility of training a Small Language Model (SLM) with 3 billion parameters to achieve expert-level performance on marine hydrodynamics problems, a domain characterised by scarce labelled data and stringent requirements for equation-based reasoning.

We propose a multi-stage pipeline comprising synthetic dataset generation, Supervised Fine-Tuning (SFT), and Reinforcement Learning with Physics-based Training (RLPT) via Group Relative Policy Optimization (GRPO). Source materials from canonical textbooks (Newman, Faltinsen) and MIT OpenCourseWare are processed through a text-extraction and OCR pipeline to produce approximately 563 text chunks. A synthetic question-answer generation system powered by the Groq API (LLaMA 3.3 70B) produces over 1,100 QA pairs spanning conceptual, numerical, and multiple-choice question types, each accompanied by a Chain-of-Thought reasoning trace.

The SFT stage fine-tunes a `Qwen2.5-3B-Instruct` model using LoRA adapters on an NVIDIA H100 GPU. A significant debugging effort was required to resolve persistent NaN gradient failures traced to BF16 precision accumulation errors across multiple LoRA target modules. The final SFT model achieves a training loss of 0.9729 and evaluation token accuracy of 77.43%.

The GRPO stage introduces a physics-based reward function combining numerical correctness verification (via SymPy), MCQ letter-matching, and a format reward to ensure non-zero intra-group advantage variance. Across three GRPO training runs, numerical accuracy improves from 14.48% (SFT baseline) to 39.37%, and MCQ accuracy improves from 60.91% to 85.45%. Critical implementation challenges including

the zero-gradient problem, TRL API incompatibilities, and a truncation bug affecting the reward signal are documented and resolved.

This work contributes a reproducible, low-resource pipeline for training domain-specific SLMs using physics-grounded reinforcement learning, and provides evidence that small models can achieve meaningful competence in specialised engineering domains.

Contents

Acknowledgements	ii
Abstract	iii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Research Objectives	2
1.4 Contributions	3
1.5 Thesis Organization	4
2 Literature Review	5
2.1 Introduction	5
2.2 Foundations of Transformer Models	5
2.2.1 Transformer Architecture Overview	5
2.3 Supervised Instruction Tuning (SFT)	6
2.4 Reinforcement Learning from Human Feedback (RLHF)	6
2.4.1 KL-Regularized RL	7
2.4.2 Policy Gradient Foundations	7
2.5 RL-Based Alignment Methods	7
2.5.1 Proximal Policy Optimization (PPO)	7
2.5.2 Group Relative Policy Optimization (GRPO)	7
2.6 Small Language Models	8
2.7 Low-Rank Adaptation (LoRA)	8
2.8 The Qwen Model Family	9
2.9 Summary	9

3	Methodology	10
3.1	Introduction	10
3.2	Data Acquisition and Text Extraction	11
3.2.1	Source Materials	11
3.2.2	Extraction Methods	12
3.3	Synthetic QA Dataset Generation	12
3.3.1	Generation Pipeline	12
3.3.2	Robustness Mechanisms	13
3.3.3	Dataset Output	13
3.4	Validation and Cleaning	13
3.5	SFT Data Preparation	14
3.6	Training Infrastructure	14
3.7	Supervised Fine-Tuning (SFT) Implementation	15
3.7.1	Base Model	15
3.7.2	LoRA Configuration	15
3.7.3	Training Configuration	16
3.7.4	Training Data	16
3.8	GRPO Implementation	16
3.8.1	Starting Point: SFT Adapter Merge	16
3.8.2	Reward Function Design	17
3.8.2.1	Correctness Reward	17
3.8.2.2	Format Reward	17
3.8.3	GRPO Hyperparameters	18
3.8.4	Physics Verifier	18
3.8.5	Evaluation Pipeline	18
4	Results and Discussion	19
4.1	Introduction	19
4.2	SFT Training Dynamics	19
4.2.1	Training Curve	19
4.2.2	SFT Summary Metrics	20
4.2.3	Qualitative SFT Verification	20
4.3	GRPO Training Results	21
4.3.1	Experimental Overview	21

4.3.2	Cross-Run Performance Comparison	21
4.3.3	Analysis of GRPO Run 1	21
4.3.4	Analysis of GRPO Run 3	21
4.3.5	Numerical Graded Breakdown	22
4.3.6	GRPO Training Dynamics	22
4.4	Cumulative Progress Summary	23
4.5	Discussion	23
4.6	Limitations	24
5	Error Analysis and Debugging	25
5.1	Overview	25
5.2	The BF16 NaN Gradient Investigation	25
5.2.1	Symptoms	25
5.2.2	Key Observation	26
5.2.3	Failed Hypotheses	26
5.2.4	Root Cause and Resolution	26
5.2.5	Key Lessons	26
5.3	TRL API Incompatibilities	27
5.4	The GRPO Zero-Gradient Problem	27
5.4.1	Symptom	27
5.4.2	Root Cause	27
5.4.3	Resolution	28
5.5	The Truncation Bug	28
5.5.1	Problem	28
5.5.2	Evidence	28
5.5.3	Resolution	28
6	Conclusion and Future Directions	29
6.1	Summary of Research	29
6.2	Synthesis of Empirical Findings	29
6.3	Implications	30
6.4	Limitations	30
6.5	Future Work	30
6.6	Concluding Remarks	31

A	Training Configuration and Hyperparameters	32
A.1	Common Configuration	32
A.2	Method-Specific Hyperparameters	33
B	Error Logs and Debugging Notes	34
B.1	Critical BF16 NaN Failure	34
B.2	Storage Exhaustion	34
B.3	GRPO Zero-Gradient Episodes	34
B.4	Truncation Bug	35
B.5	Physics Verifier IndexError	35
B.6	Resolution Summary Table	35
	Bibliography	35

List of Figures

2.1	Decoder-only transformer block structure.	6
2.2	GRPO training loop: group-normalized advantages eliminate the value network.	8
3.1	End-to-end pipeline for the Marine Hydrodynamics SLM.	11

List of Tables

3.1	Text extraction results by source.	12
3.2	Hardware and software environment.	14
3.3	Final LoRA hyperparameters for SFT.	15
3.4	Final SFT training hyperparameters.	16
3.5	Format reward components (Run 3 weights).	17
3.6	GRPO hyperparameters across runs.	18
4.1	SFT training log (selected checkpoints).	20
4.2	Evaluation results across all training stages (n=552).	21
4.3	Numerical accuracy graded breakdown for GRPO Run 3 (n=221). . .	22
4.4	Within-training learning progression (Run 3).	22
4.5	Cumulative improvement across all training stages.	23
5.1	Summary of failed hypotheses during NaN debugging.	26
5.2	TRL API errors and resolutions.	27
A.1	Hyperparameters for SFT and GRPO training.	33
B.1	Summary of major errors and their resolutions.	35

Chapter 1

Introduction

1.1 Background

Recent developments in Large Language Models (LLMs) have demonstrated remarkable capabilities in reasoning, coding, and natural language understanding. Models with hundreds of billions of parameters can perform sophisticated multi-step reasoning and generate coherent, contextually appropriate responses across diverse domains. However, these models are computationally expensive and often inefficient for domain-specific tasks requiring structured numerical reasoning and deep subject-matter expertise.

Recent research indicates that Small Language Models (SLMs) with 1–8 billion parameters, when fine-tuned on specialized datasets, can match or outperform larger models on specific tasks while requiring significantly lower computational resources (Hu et al. 2025). Surveys of recent work confirm that task-specific SLMs frequently rival or exceed larger models on focused benchmarks such as reasoning, structured output generation, and domain-specific question answering (Red Hat AI 2024).

This thesis explores the use of SLMs trained with Reinforcement Learning with Physics-based Training (RLPT) via Group Relative Policy Optimization (GRPO) to solve marine hydrodynamics problems. The goal is to demonstrate that a small model with 3 billion parameters can achieve comparable or better performance than general-purpose LLMs on hydrodynamics questions, leveraging synthetic dataset generation and physics-based reward verification to enable reinforcement learning in a data-scarce domain.

1.2 Problem Statement

Hydrodynamics is a specialised engineering domain involving problems related to potential flow theory, wave-body interactions, ship hydrodynamics, boundary layer theory, and fluid forces and hydrodynamic coefficients. While general-purpose LLMs possess broad knowledge, they often perform poorly on specialised engineering problems requiring structured reasoning and equation-based solutions.

Three key challenges motivate this work:

1. **Lack of labelled datasets:** Very few publicly available question-answer datasets exist for marine hydrodynamics. Unlike natural language understanding or code generation, this domain has no established benchmark.
2. **Requirement for numerical and symbolic reasoning:** Many problems require equation derivations, numerical calculations, and multi-step reasoning that go beyond pattern matching.
3. **Evaluation difficulty:** Solutions must be verified using physical laws and equations rather than subjective evaluation, necessitating automated physics-based verification.

This thesis investigates whether a domain-specialised SLM trained using reinforcement learning with verifiable physics-based rewards can solve hydrodynamics problems more accurately than general-purpose approaches.

1.3 Research Objectives

The primary objectives of this project are:

1. **To develop a task-specific Small Language Model** for marine hydrodynamics problem solving.
2. **To create a hydrodynamics dataset** using synthetic data generation techniques from canonical textbooks and course materials.
3. **To design a physics-based reward verification system** for reinforcement learning that combines numerical comparison, symbolic equivalence checking, and structural format verification.

4. **To train the model using Supervised Fine-Tuning (SFT) followed by GRPO-based reinforcement learning**, establishing a complete two-stage alignment pipeline.
5. **To document the complete engineering process**, including all debugging challenges, failure modes, and their resolutions, to aid reproducibility.

1.4 Contributions

This thesis makes the following contributions:

- **Marine Hydrodynamics QA Dataset:** A synthetic dataset of over 1,100 question-answer pairs with Chain-of-Thought reasoning, spanning conceptual, numerical, and multiple-choice question types, generated from authoritative textbooks and MIT OpenCourseWare materials.
- **Domain-Specific SLM Training Pipeline:** A complete, reproducible pipeline from text extraction and OCR through synthetic QA generation, SFT, and GRPO training, all operating on a single GPU server.
- **Physics-Grounded Reward Function:** A multi-component reward function for GRPO that combines numerical correctness verification (with graded partial credit), MCQ letter-matching, and format rewards to ensure effective policy gradient training.
- **Detailed Error Analysis:** Comprehensive documentation of critical debugging challenges including BF16 NaN gradient failures, TRL API incompatibilities, the GRPO zero-gradient problem, and a truncation bug affecting reward signals.
- **Empirical Results:** Evidence that GRPO training improves numerical accuracy from 14.48% to 39.37% and MCQ accuracy from 60.91% to 85.45% over the SFT baseline.

1.5 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a literature review of Small Language Models, LLM alignment techniques, reinforcement learning for reasoning, and parameter-efficient fine-tuning methods.
- **Chapter 3** describes the methodology, covering the data pipeline from text extraction through synthetic QA generation, the SFT training process, and the GRPO reinforcement learning implementation.
- **Chapter 4** presents the experimental results across all training stages, including SFT convergence analysis, GRPO training dynamics across three runs, and evaluation metrics.
- **Chapter 5** provides a dedicated analysis of errors and debugging, documenting the NaN investigation, API incompatibilities, and reward-signal corruption.
- **Chapter 6** concludes the thesis and outlines future research directions.

Chapter 2

Literature Review

2.1 Introduction

The rapid progress in large language models has intensified the need for reliable methods to specialise these models for domain-specific tasks. Although modern LLMs demonstrate strong zero-shot and instruction-following capabilities, these behaviours do not naturally extend to specialised scientific and engineering domains that require precise numerical reasoning. This chapter reviews the foundational methods underlying this thesis, covering transformer architectures, supervised fine-tuning, reinforcement learning from human feedback, and parameter-efficient adaptation techniques.

2.2 Foundations of Transformer Models

Transformer architectures, first proposed by Vaswani et al. (2017), form the basis for nearly all state-of-the-art language models. Unlike recurrent or convolution-based models, transformers rely entirely on self-attention mechanisms to capture long-range dependencies, allowing them to scale to billions of parameters while maintaining efficient parallelism during training.

2.2.1 Transformer Architecture Overview

The standard decoder-only transformer, widely used in LLMs, consists of stacked blocks each containing multi-head self-attention (MHA), a feed-forward network (FFN), layer normalization, and residual connections.

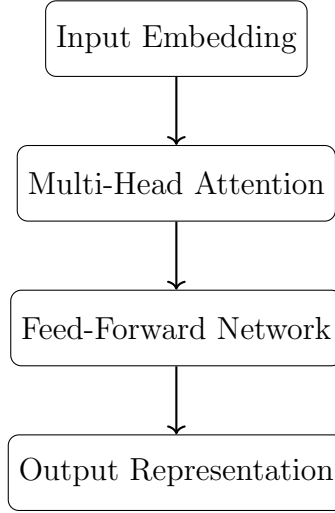


Figure 2.1: Decoder-only transformer block structure.

Multi-head attention computes weighted representations using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (2.1)$$

2.3 Supervised Instruction Tuning (SFT)

Supervised fine-tuning is typically the first step in adapting an LLM for a target domain. During SFT, the model is trained on high-quality instruction–response pairs that demonstrate desirable behaviours. Given input–output pairs (x, y) , SFT minimizes cross-entropy loss:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y)}[\log \pi_{\theta}(y|x)]. \quad (2.2)$$

SFT establishes a foundational alignment layer, making the model’s initial behaviour stable enough for more delicate preference-based or reinforcement-learning-based optimization. However, SFT alone cannot guarantee accurate numerical reasoning, which motivates the subsequent RL stage.

2.4 Reinforcement Learning from Human Feedback (RLHF)

RLHF has emerged as a key component of LLM alignment pipelines, used in models such as InstructGPT (Ouyang et al. 2022). RLHF introduces a reward model trained

on human preference data, which assigns a scalar reward to model outputs. The policy is then optimized via reinforcement learning to maximize expected reward.

2.4.1 KL-Regularized RL

To prevent the model from drifting too far from the reference policy, KL regularization is added:

$$\max_{\theta} \mathbb{E} [r_{\phi}(x, y) - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})] . \quad (2.3)$$

2.4.2 Policy Gradient Foundations

RLHF ultimately relies on the policy gradient theorem (Sutton et al. 1999):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) A(s, a)], \quad (2.4)$$

where $A(s, a)$ is the advantage function.

2.5 RL-Based Alignment Methods

2.5.1 Proximal Policy Optimization (PPO)

PPO (Schulman et al. 2017) introduces a clipping mechanism to prevent large policy updates:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E} [\min(r_{\theta} A, \text{clip}(r_{\theta}, 1 - \epsilon, 1 + \epsilon) A)] , \quad (2.5)$$

where $r_{\theta} = \pi_{\theta}(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. PPO is memory-intensive, requiring three models simultaneously (policy, reference, value), making it impractical for resource-constrained settings.

2.5.2 Group Relative Policy Optimization (GRPO)

GRPO was introduced in the DeepSeekMath work (Shao et al. 2024). It removes the need for a value network by normalizing advantages across groups of samples:

$$A_i^G = A_i - \frac{1}{|G|} \sum_{j \in G} A_j. \quad (2.6)$$

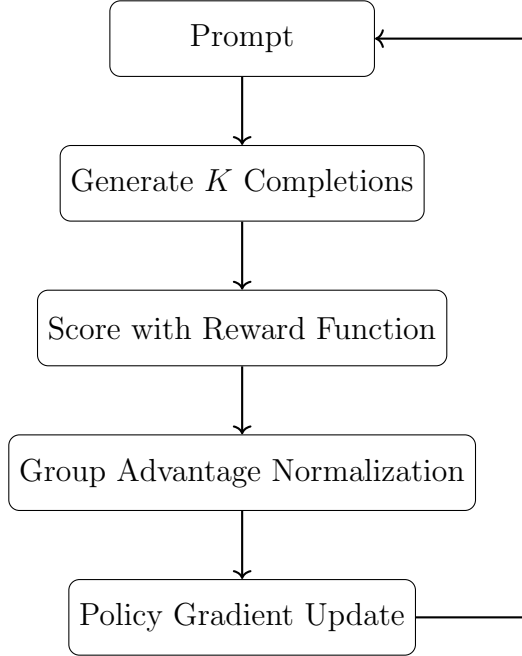


Figure 2.2: GRPO training loop: group-normalized advantages eliminate the value network.

By eliminating the value model, GRPO reduces VRAM requirements and improves training stability, making it particularly suited for low-resource reinforcement learning. GRPO is central to the RLPT approach used in this thesis.

2.6 Small Language Models

Recent research has demonstrated that task-specialised small models (1–8B parameters) can match or exceed larger models on structured tasks (Hu et al. 2025). SLMs fine-tuned for structured output generation have achieved over 98% formatting accuracy, outperforming larger models on constrained decoding tasks (Red Hat AI 2024). These findings suggest that specialised SLMs can be highly effective in narrow domains such as engineering and scientific problem solving.

2.7 Low-Rank Adaptation (LoRA)

LoRA (Hu et al. 2022) freezes base model weights and introduces trainable low-rank decomposition matrices:

$$W' = W + BA, \tag{2.7}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $\text{rank } r \ll \min(d, k)$. This dramatically reduces the number of trainable parameters while preserving the capacity for task-specific adaptation.

2.8 The Qwen Model Family

Qwen2.5 (Yang et al. 2024) is an open-source language model series developed by Alibaba. The **Qwen2.5-3B-Instruct** variant used in this thesis features a decoder-only transformer with RoPE positional embeddings, SwiGLU feed-forward networks, and has been instruction-tuned on high-quality supervised data. Its 3-billion parameter size provides a strong balance between capability and computational efficiency, making it well-suited for domain-specific fine-tuning on a single GPU.

2.9 Summary

This chapter reviewed the technical foundations underlying this thesis. We covered transformer architectures, supervised fine-tuning, RLHF and its policy optimization variants (PPO, GRPO), the emerging capabilities of Small Language Models, and parameter-efficient fine-tuning via LoRA. Together, these methods provide the conceptual and practical framework for the domain-specific training pipeline implemented in subsequent chapters.

Chapter 3

Methodology

3.1 Introduction

This chapter describes the complete methodology used to build a physics-aware Small Language Model for marine hydrodynamics. The pipeline spans five stages: data acquisition and text extraction, synthetic QA dataset generation, data validation and cleaning, Supervised Fine-Tuning (SFT), and Group Relative Policy Optimization (GRPO). The goal is to transform a general-purpose 3B parameter language model into a domain specialist capable of solving conceptual, numerical, and multiple-choice hydrodynamics problems.

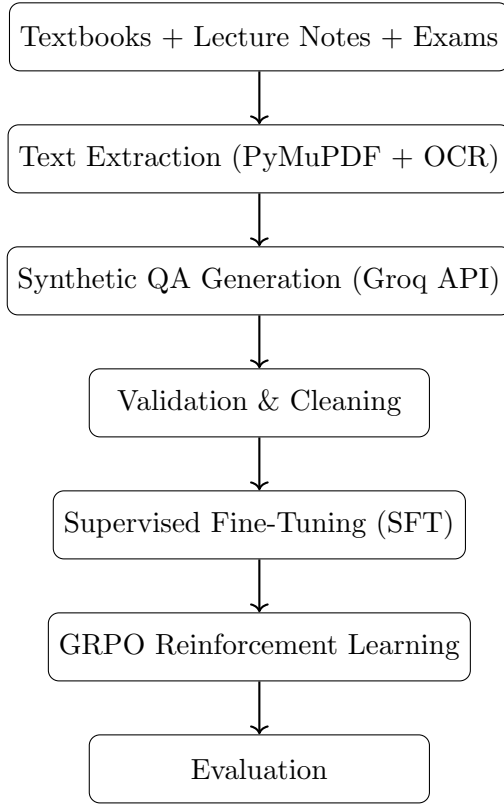


Figure 3.1: End-to-end pipeline for the Marine Hydrodynamics SLM.

3.2 Data Acquisition and Text Extraction

3.2.1 Source Materials

The training data is derived from authoritative sources in marine hydrodynamics:

- **Newman, J.N.** — *Marine Hydrodynamics* (digital text, primary source)
- **Faltinsen, O.M.** — *Sea Loads on Ships and Offshore Structures* (OCR-scanned)
- **MIT OCW 2.20** — Marine Hydrodynamics: 22 lecture notes, 25 problem sets, final exam
- **MIT OCW 2.29** — Numerical Marine Hydrodynamics: 14 lecture notes
- **IIT Exam Papers** — 4 graduate-level examination papers

In total, approximately 69 PDFs were collected spanning theoretical explanations, derivations, worked examples, and examination problems.

3.2.2 Extraction Methods

Two extraction methods were employed depending on the source format:

- **Digital PDFs** (Newman textbook, MIT OCW lecture notes and problem sets):
Processed using PyMuPDF for direct text extraction.
- **Scanned PDFs** (Faltinsen textbook, MIT OCW 2.29 notes, IIT exam papers):
Processed using Tesseract OCR with PyMuPDF page rendering.

Table 3.1: Text extraction results by source.

Source	Output	Method
Newman textbook	340 chunks	PyMuPDF
Faltinsen textbook	156 chunks	OCR
MIT OCW 2.20 lectures	22 files	PyMuPDF
MIT OCW 2.20 problem sets	25 files	PyMuPDF
MIT OCW 2.20 final exam	1 file	PyMuPDF
MIT OCW 2.29 lecture notes	14 files	OCR
IIT exam papers	4 files	OCR
Total	563 files	2.4 MB

3.3 Synthetic QA Dataset Generation

3.3.1 Generation Pipeline

Due to the absence of publicly available QA datasets for marine hydrodynamics, a synthetic generation pipeline was developed using the script `generate_qa.py`. Each extracted text chunk is provided as context to a large language model, which generates graduate-level questions and detailed step-by-step solutions.

- **Model:** LLaMA 3.3 70B via the Groq API
- **Prompt Template:** Loaded from `qa_prompt_template.txt`, requesting conceptual, numerical, and MCQ question types
- **Output Format:** Each answer includes a Chain-of-Thought (CoT) reasoning trace

3.3.2 Robustness Mechanisms

The generation script implements several robustness features:

- **API Key Rotation:** Multiple Groq API keys are rotated to handle HTTP 429 rate-limit errors.
- **Checkpointing:** Progress is tracked in `qa_checkpoint.json` to allow resumption after interruptions.
- **JSON Extraction:** Regex-based parsing pulls structured JSON blocks from model responses.

3.3.3 Dataset Output

The generation pipeline produced approximately 1,100 QA pairs stored in `qa_dataset.jsonl`. Each record contains: question text, answer, chain-of-thought reasoning, question type (conceptual, numerical, or MCQ), options (for MCQs), and source chunk filename.

3.4 Validation and Cleaning

The validation script (`validate_qa.py`) applies the following filters:

- Removal of “refusal” samples (e.g., “I cannot answer this”)
- Removal of “context leakage” phrases (e.g., “According to the passage”)
- Minimum word count enforcement (conceptual answers > 30 words, CoT > 20 words)
- Unicode cleaning: replacement of the ∞ symbol with the word “infinity” to prevent BF16 numerical instability
- Structural integrity checks (all required fields present)

The cleaned dataset (`qa_dataset_clean.jsonl`) is then split into training (90%, 976 records) and evaluation (10%, 109 records) sets.

3.5 SFT Data Preparation

The script `prepare_sft_data.py` converts validated QA pairs into the HuggingFace ChatML format compatible with TRL 1.0.0+. Each record consists of a system prompt, user message (question), and assistant response containing the Chain-of-Thought followed by the Final Answer.

System Prompt:

You are an expert AI assistant specializing in Marine Hydrodynamics and Ocean Engineering. Approach all questions methodically and provide step-by-step reasoning.

Assistant Response Format:

Chain of Thought:

[step-by-step reasoning]

Final Answer:

[answer]

An earlier design using `<think>...</think>` XML tags was abandoned after discovering that custom special tokens triggered NaN gradient spikes during BF16 training (see Chapter 5 for the full debugging analysis).

3.6 Training Infrastructure

Table 3.2: Hardware and software environment.

Component	Specification
Server	mtp-server (10.71.9.8)
GPU	NVIDIA H100 80 GB SXM5
RAM	~126 GB
OS	Ubuntu 24.04 LTS
Python	3.12
PyTorch	2.x (CUDA 12.x)
Transformers	4.x
TRL	1.0.0
PEFT	0.x
Accelerate	0.x

Storage constraint: The `/home` partition was at 100% capacity throughout training. Model checkpoints and the HuggingFace cache were redirected to `/tmp` (RAM-backed tmpfs storage).

3.7 Supervised Fine-Tuning (SFT) Implementation

3.7.1 Base Model

The base model is `Qwen/Qwen2.5-3B-Instruct`, a 3-billion parameter decoder-only transformer with RoPE embeddings and SwiGLU activations.

3.7.2 LoRA Configuration

Table 3.3: Final LoRA hyperparameters for SFT.

Parameter	Value
Rank (r)	16
LoRA Alpha	16 (scaling = 1.0)
Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj
Bias	None
Task Type	CAUSAL_LM

The initial configuration targeted 7 modules (including `gate_proj`, `up_proj`, `down_proj`) with a scaling factor of 2.0. This was reduced to 4 attention-only modules with scaling 1.0 as part of the NaN debugging resolution (see Chapter 5).

3.7.3 Training Configuration

Table 3.4: Final SFT training hyperparameters.

Parameter	Value
Batch size (per device)	4
Gradient accumulation steps	4 (effective batch size = 16)
Epochs	3
Learning rate	5×10^{-5}
Precision	Float32 (full precision)
Optimizer	AdamW (torch)
Max gradient norm	1.0
Warmup steps	10

The critical decision to train in Float32 rather than BF16 resolved persistent NaN gradient failures. The H100’s 80 GB VRAM comfortably accommodates Float32 training of a 3B model with LoRA adapters (~ 36 GB total memory usage).

3.7.4 Training Data

The SFT training set consists of 976 ChatML-formatted records and the evaluation set contains 109 records, split with `random.seed(42)`.

3.8 GRPO Implementation

3.8.1 Starting Point: SFT Adapter Merge

GRPO training does not start from the bare pretrained model. Instead:

1. Load `Qwen/Qwen2.5-3B-Instruct` base weights.
2. Load the SFT LoRA adapter from `checkpoint-183`.
3. Merge the SFT adapter into the base weights via `model.merge_and_unload()`.
4. Apply a new LoRA adapter (identical architecture) for GRPO training.

This ensures GRPO builds on the domain knowledge established during SFT.

3.8.2 Reward Function Design

The reward function is the core innovation of the GRPO stage. It consists of two components:

3.8.2.1 Correctness Reward

For numerical questions, a graded partial-credit function (introduced in Run 2) replaces the initial binary reward:

$$r_{\text{num}}(\hat{y}, y^*) = \begin{cases} 1.0 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.05 \\ 0.7 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.10 \\ 0.4 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.25 \\ 0.15 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.50 \\ 0.0 & \text{otherwise} \end{cases} \quad (3.1)$$

For MCQ questions, the reward is binary: 1.0 for a correct letter match, 0.0 otherwise.

3.8.2.2 Format Reward

A multi-component format reward provides continuous signal to prevent the zero-gradient problem inherent in purely binary rewards:

Table 3.5: Format reward components (Run 3 weights).

Component	Weight	Criterion
Reasoning section	0.05	Contains “Chain of Thought” or “step-by-step”
Final Answer line	0.08	Contains “Final Answer:”
Reasonable length	0.05	50–2000 characters
Structured steps	0.02	Numbered steps or bullet points
Max total	~0.20	

The total reward per completion is: $r = r_{\text{correctness}} + r_{\text{format}}$.

3.8.3 GRPO Hyperparameters

Table 3.6: GRPO hyperparameters across runs.

Parameter	Run 1	Run 2	Run 3
Epochs	1	2	2
Learning rate	1×10^{-5}	2×10^{-5}	2×10^{-5}
Num. generations (K)	8	6	8
Max new tokens	768	512	1024 [†]
Temperature	0.8	0.8	0.8
Top- p	0.95	0.95	0.95
Batch size	1	1	1
Grad. accumulation	4	4	4
Precision	Float32	Float32	Float32

[†]Did not take effect during training due to TRL API bug; see Section 5.4.

3.8.4 Physics Verifier

The `physics_verifier.py` module provides automated evaluation:

- **Numerical verification:** Extracts numerical answers from model output and compares against ground truth with configurable tolerance (default 5%).
- **Symbolic verification:** Uses SymPy for symbolic equivalence checking of equations.
- **MCQ verification:** Extracts the selected letter using robust pattern matching (handles formats like “Answer: B”, “Correct option is (C)”, “\boxed{A}”).

3.8.5 Evaluation Pipeline

Dedicated evaluation scripts (`eval_baseline.py` and `eval_grpo.py`) compute per-item and aggregate metrics on a held-out evaluation set of 552 records. Training and evaluation datasets are verified to be completely disjoint.

Chapter 4

Results and Discussion

4.1 Introduction

This chapter presents the experimental results across all training stages. We begin with the SFT training dynamics, followed by the GRPO results across three progressive training runs, and conclude with a comparative analysis and discussion of the findings.

4.2 SFT Training Dynamics

4.2.1 Training Curve

The final SFT training run completed 3 epochs over 976 training records without any NaN gradient failures (the resolution is documented in Chapter 5). The training curve demonstrates steady convergence:

Table 4.1: SFT training log (selected checkpoints).

Epoch	Train Loss	Grad Norm	Token Acc.
0.164	2.193	0.3448	63.2%
0.328	1.767	0.2905	65.2%
0.492	1.348	0.2137	69.8%
0.656	1.103	0.1553	73.9%
0.820	0.996	0.1008	76.5%
0.984	0.932	0.0994	77.4%
<i>Eval @ Epoch 1: Loss = 1.035, Accuracy = 75.5%</i>			
1.475	0.775	0.0585	80.0%
1.967	0.770	0.0580	79.7%
<i>Eval @ Epoch 2: Loss = 0.983, Accuracy = 77.0%</i>			
2.459	0.731	0.0459	80.8%
2.951	0.745	0.0514	80.4%
<i>Eval @ Epoch 3: Loss = 0.974, Accuracy = 77.4%</i>			

4.2.2 SFT Summary Metrics

- **Final training loss:** 0.9729
- **Final evaluation loss:** 0.9743
- **Evaluation token accuracy:** 77.43%
- **Training runtime:** 564.6 seconds (~ 9.4 minutes for 3 epochs)

The model shows clear convergence with a small train–eval gap (0.0014), suggesting minimal overfitting. The decreasing gradient norm indicates stable optimization throughout training.

4.2.3 Qualitative SFT Verification

Smoke-test inference confirmed that the SFT model produces physically correct conceptual explanations (e.g., Reynolds number definitions, laminar–turbulent transition) and can perform numerical calculations using correct formulas. However, numerical accuracy on the held-out evaluation set was only 14.48%, motivating the GRPO stage.

4.3 GRPO Training Results

4.3.1 Experimental Overview

Three GRPO training runs were conducted, each introducing improvements to the reward function and training configuration:

- **Run 1:** Binary correctness reward + format reward; 1 epoch, $K = 8$.
- **Run 2:** Graded numerical reward; reduced format weights; completion logging; 2 epochs.
- **Run 3:** Extended training with 2 epochs, $K = 8$; `max_new_tokens` set to 1024 (but not applied during training due to a TRL API bug).

4.3.2 Cross-Run Performance Comparison

Table 4.2: Evaluation results across all training stages (n=552).

Metric	SFT Baseline	GRPO Run 1	GRPO Run 3	Δ (Run 3 – SFT)
Numerical accuracy	14.48%	38.01%	39.37%	+24.89 pp
MCQ accuracy	60.91%	75.45%	85.45%	+24.54 pp

4.3.3 Analysis of GRPO Run 1

Run 1 achieved the largest single-run improvement: numerical accuracy nearly tripled from 14.48% to 38.01% (+23.53 pp), and MCQ accuracy improved by 14.55 pp. The format reward successfully encouraged structured reasoning, and the binary correctness reward provided sufficient signal for initial policy improvement.

4.3.4 Analysis of GRPO Run 3

Run 3 completed all 5,300 optimizer steps (11 hours 54 minutes) without crashes. Training produced 21,200 logged completions (2,650 prompts $\times$ 8 generations).

MCQ is the standout result: 85.45% accuracy represents a +10 pp improvement over Run 1 and +24.54 pp over the SFT baseline. The model shows consistent improvement in MCQ performance across training.

Numerical improvement was marginal (39.37% vs. 38.01%), limited by the truncation bug that corrupted the reward signal during training (see Section 5.4).

4.3.5 Numerical Graded Breakdown

Table 4.3: Numerical accuracy graded breakdown for GRPO Run 3 (n=221).

Bucket	Count	%	Meaning
Exact ($\leq 5\%$ error)	43	19.5%	Binary PASS
Near ($\leq 10\%$ error)	0	0.0%	—
Close ($\leq 25\%$ error)	5	2.3%	Reasonable
Far ($\leq 50\%$ error)	21	9.5%	Right ballpark
Wrong ($> 50\%$ or no match)	152	68.8%	Completely off
No number extracted	3	1.4%	No parseable output

The bimodal distribution (68.8% completely wrong, 19.5% exact) suggests that failures are primarily due to incorrect formula application rather than arithmetic imprecision. The model either applies the correct formula and gets the right answer, or uses an entirely wrong approach.

4.3.6 GRPO Training Dynamics

The GRPO training loss is a clipped policy-gradient surrogate objective and is expected to be negative when the policy assigns high probability to high-advantage completions. The final training loss of -0.005328 indicates good alignment between the policy and the reward signal.

Learning progression from completions log (Run 3):

Table 4.4: Within-training learning progression (Run 3).

Period	MCQ Correct	Numerical Exact
First 25% of training	80.1%	16.4%
Last 25% of training	85.3%	15.5%
Change	+5.2 pp	−0.9 pp (flat)

MCQ showed clear within-training improvement. Numerical accuracy remained flat during training due to the truncation bug corrupting the numerical reward signal (75–95% of completions were truncated by end of training).

4.4 Cumulative Progress Summary

Table 4.5: Cumulative improvement across all training stages.

Stage	Num. Acc.	MCQ Acc.	Key Contribution
SFT Baseline	14.48%	60.91%	Domain knowledge, format structure
GRPO Run 1	38.01%	75.45%	RL reward signal, format reward
GRPO Run 3	39.37%	85.45%	Extended training, refined rewards

4.5 Discussion

The results demonstrate that reinforcement learning with physics-based rewards can substantially improve a domain-specific SLM’s accuracy beyond what SFT alone achieves. Several key observations emerge:

1. **SFT provides domain knowledge but not numerical precision.** The SFT model achieves 77.4% token accuracy and produces well-structured responses with correct domain terminology, but its numerical accuracy (14.48%) is insufficient for practical use.
2. **GRPO is effective for MCQ improvement.** The progression from 60.91% to 85.45% across three GRPO runs demonstrates that the combination of binary correctness reward and structured format reward is effective for multiple-choice reasoning.
3. **Numerical accuracy requires careful reward engineering.** The initial jump from 14.48% to 38.01% in Run 1 was substantial, but subsequent runs showed diminishing returns, partly due to the truncation bug and partly due to the bimodal nature of numerical failures (correct formula vs. completely wrong approach).
4. **The truncation bug was the primary bottleneck.** The discovery that `max_new_tokens` was not being applied during GRPO training explains the stagnation of numerical accuracy between Runs 1 and 3. With 75–95% of completions truncated before reaching “Final Answer:”, the numerical reward signal was severely corrupted.

4.6 Limitations

- **Conceptual evaluation not performed:** Due to the lack of access to paid API keys or a sufficient number of free API keys, the LLM-as-judge evaluation for conceptual questions could not be executed. Conceptual accuracy remains unmeasured.
- **Truncation bug in GRPO Runs 1–3:** The `max_new_tokens` parameter was not applied during training rollouts due to a TRL API incompatibility, corrupting the numerical reward signal.
- **Single GPU:** All experiments were conducted on a single H100 GPU, limiting the scale of hyperparameter searches.
- **Dataset size:** The synthetic dataset of $\sim 1,100$ QA pairs is modest compared to standard NLP benchmarks, though it is substantial for a niche engineering domain.

Chapter 5

Error Analysis and Debugging

5.1 Overview

This chapter documents the major technical challenges encountered during the project and their resolutions. The debugging process consumed a significant portion of the project timeline and yielded insights that are valuable for reproducibility. We categorize the errors into four groups: BF16 NaN gradient failures, TRL API incompatibilities, the GRPO zero-gradient problem, and the truncation bug.

5.2 The BF16 NaN Gradient Investigation

5.2.1 Symptoms

Every SFT training run with BF16 precision would begin normally (2–5 steps of valid decreasing loss) and then catastrophically produce `grad_norm: nan`, permanently destroying the model weights:

```
{ 'loss': '2.296', 'grad_norm': '0.9745', 'epoch': '0.16' } OK
{ 'loss': '1.318', 'grad_norm': '0.1923', 'epoch': '0.33' } OK
{ 'loss': '1.052', 'grad_norm': '0.1228', 'epoch': '0.49' } OK
{ 'loss': '0.960', 'grad_norm': '0.1199', 'epoch': '0.66' } OK
{ 'loss': '1.124', 'grad_norm': 'nan',      'epoch': '0.82' } FAIL
{ 'loss': '0',      'grad_norm': 'nan',      'epoch': '0.98' } DEAD
```

5.2.2 Key Observation

The NaN always hit at exactly the same optimizer step (step 50) with `random.seed(42)`, producing identical loss values across all runs. This perfect reproducibility ruled out stochastic numerical instability and confirmed a structural training issue.

5.2.3 Failed Hypotheses

Table 5.1: Summary of failed hypotheses during NaN debugging.

#	Hypothesis	Change Made	Result
1	<think> tokenization	Added as special tokens	NaN persisted
2	Random embedding init	Initialized with mean	NaN persisted
3	Random lm_head init	Initialized with mean	NaN persisted
4	Fused AdamW bug	Changed to <code>adamw_torch</code>	NaN persisted
5	∞ in data	Replaced with “infinity”	NaN persisted
6	Bad batch clustering	Reshuffled with seed=1337	NaN at step 1
7	<think> unseen token	Removed tags entirely	NaN at step 3

5.2.4 Root Cause and Resolution

The root cause was **BF16 precision accumulation errors across multiple LoRA matrices**. BF16’s limited 7-bit mantissa accumulates rounding errors during the optimizer step when many LoRA matrices are updated simultaneously. The number of steps before failure is roughly proportional to the number of active LoRA matrices and their scaling factor.

Resolution: Switching to Float32 training eliminated all NaN failures. Additionally, LoRA targets were reduced from 7 modules (14 matrices) to 4 attention-only modules (8 matrices) and the scaling factor was halved from 2.0 to 1.0.

5.2.5 Key Lessons

1. `loss: 0` alongside `grad_norm: nan` is the signature of a dead model, not perfect convergence.
2. BF16 is dangerous for LoRA fine-tuning when targeting many modules simultaneously.

3. Perfectly reproducible NaN indicates a structural issue, not a stochastic one. Different random seeds move the failure to a different step but do not eliminate it.

5.3 TRL API Incompatibilities

The TRL library (version 1.0.0) exhibited several API inconsistencies that required iterative debugging:

Table 5.2: TRL API errors and resolutions.

Error	Resolution
<code>SFTTrainer.__init__()</code> got unexpected keyword argument <code>dataset_text_field</code>	Rewrote script to use <code>SFTConfig</code> and <code>ChatML messages</code> format
<code>GRPOConfig</code> dropped <code>max_new_tokens</code>	Implemented <code>_safe_config_kwargs</code> helper to inspect <code>GRPOConfig.__init__</code> signature at runtime and apply unsupported parameters via <code>model.generation_config</code> fallback
<code>TypeError: unhashable type: 'set'</code> with <code>device_map="auto"</code>	Loaded base model to explicit device before applying PEFT adapter

5.4 The GRPO Zero-Gradient Problem

5.4.1 Symptom

Initial GRPO runs showed `grad_norm = 0` for a high percentage of training steps.

5.4.2 Root Cause

GRPO computes advantages by comparing rewards within a group of K completions. When all K completions receive the same reward (e.g., all 0.0 for binary correctness), the advantage for every completion is zero, producing zero gradients.

5.4.3 Resolution

Three fixes were applied iteratively:

1. **Increased K from 4 to 8:** More completions per group increases the probability of within-group reward variance.
2. **Added sampling diversity:** `temperature=0.8` and `top_p=0.95` replaced near-greedy generation.
3. **Non-binary format reward:** The format reward (Section 3.8.2) provides continuous reward variation even when all completions have the same correctness score. This was the most impactful fix.

5.5 The Truncation Bug

5.5.1 Problem

The `max_new_tokens` parameter was passed to `GRPOConfig`, but TRL 1.0.0 does not accept it. The `_safe_config_kwargs` helper correctly detected and dropped it, but unlike `temperature` and `top_p` (which had fallback code), there was no fallback for `max_new_tokens`. The model defaulted to its built-in limit of ~ 256 tokens.

5.5.2 Evidence

Training metrics showed `completions/clipped_ratio` rising from 0.05–0.15 early in training to 0.75–0.95 late in training, with `completions/max_length` capped at 256. The model learned to produce longer answers but was hard-capped, truncating before “Final Answer:” and receiving false zero rewards.

5.5.3 Resolution

A `model.generation_config` fallback was added for `max_new_tokens`, matching the existing pattern for `temperature` and `top-p`. This fix is applied in Run 4 (planned).

Chapter 6

Conclusion and Future Directions

6.1 Summary of Research

This thesis investigated the feasibility of training a Small Language Model for expert-level reasoning in marine hydrodynamics using a combination of Supervised Fine-Tuning and Reinforcement Learning with Physics-based Training. Starting from a general-purpose 3B parameter model (`Qwen2.5-3B-Instruct`), we built a complete pipeline spanning data extraction from canonical textbooks, synthetic QA dataset generation, SFT with LoRA adapters, and GRPO-based reinforcement learning with a physics-grounded reward function.

6.2 Synthesis of Empirical Findings

The experimental results yield three key conclusions:

1. **SFT establishes domain knowledge but not numerical precision.** The SFT model achieved 77.4% token accuracy and strong conceptual understanding, but only 14.48% numerical accuracy. SFT alone is insufficient for engineering problem-solving.
2. **GRPO substantially improves task-specific accuracy.** Across three training runs, numerical accuracy improved to 39.37% (+24.89 pp) and MCQ accuracy reached 85.45% (+24.54 pp), demonstrating that physics-based rewards provide effective training signal for reinforcement learning.

3. **Reward engineering is critical.** The progression from binary rewards (Run 1) to graded partial-credit rewards (Run 2) and the identification of the truncation bug (Run 3) highlight that the design and debugging of reward functions is as important as the choice of RL algorithm.

6.3 Implications

This work demonstrates that Small Language Models can achieve meaningful competence in specialised engineering domains when trained with appropriate reward signals. The combination of synthetic data generation and physics-based reward verification provides a template for adapting language models to other scientific and engineering domains where labelled data is scarce.

6.4 Limitations

- **Conceptual evaluation:** Due to the lack of access to paid LLM API keys (or a large number of free keys), the LLM-as-judge evaluation for conceptual questions was not performed. Conceptual accuracy remains unmeasured.
- **Truncation bug:** The `max_new_tokens` parameter was not applied during GRPO training in Runs 1–3, corrupting the numerical reward signal and limiting numerical accuracy improvement.
- **Dataset scale:** The synthetic dataset of $\sim 1,100$ QA pairs, while substantial for the domain, limits the statistical power of convergence analysis.
- **Single evaluation metric:** Reliance on accuracy and graded reward, while sufficient for feasibility analysis, lacks the rigour of large-scale automated benchmarks.

6.5 Future Work

Building on the results established in this thesis, the following directions are planned or recommended:

1. **GRPO Run 4 (Planned):** With the `max_new_tokens` fallback fix applied, Run 4 is expected to show substantial improvement in numerical accuracy. The fix ensures that the 1024-token generation limit takes effect during training rollouts, allowing the model to produce complete answers and receive accurate reward signals. The verification checklist includes confirming `completions/clipped_ratio` stays below 0.3 in early training steps.
2. **Conceptual Evaluation:** Once API access is available, the LLM-as-judge pipeline should be activated to measure conceptual accuracy and provide a complete picture of model performance across all question types.
3. **VLLM Integration:** Serving the model via VLLM for high-throughput RL rollouts would significantly reduce per-step training time, enabling more epochs and larger generation groups.
4. **Curriculum-Based Training:** Structuring GRPO training to progress from simpler to harder numerical problems may improve the model’s ability to learn correct formula application.
5. **Hybrid Reward Modelling:** Investigating hybrid reward functions that combine physics verification with a small, quantized neural reward model could provide richer training signal, particularly for conceptual questions.
6. **Comparison with Larger Models:** Evaluating the trained SLM against GPT-4 class models and other open-source LLMs on the same evaluation set would provide external validation of the approach.

6.6 Concluding Remarks

This thesis establishes that Supervised Fine-Tuning combined with GRPO-based Reinforcement Learning provides a viable path for training domain-specific Small Language Models in data-scarce engineering domains. While numerical accuracy requires further improvement through the resolution of the truncation bug and continued training, the MCQ accuracy of 85.45% and the overall pipeline design demonstrate that physics-grounded reinforcement learning is a promising approach for specialising language models in scientific and engineering disciplines.

Appendix A

Training Configuration and Hyperparameters

A.1 Common Configuration

- **Base Model:** Qwen/Qwen2.5-3B-Instruct
- **Precision:** Float32 (full precision)
- **LoRA Rank (r):** 16
- **LoRA Alpha:** 16 (scaling = 1.0)
- **Target Modules:** q_proj, k_proj, v_proj, o_proj
- **Optimizer:** AdamW (torch)

A.2 Method-Specific Hyperparameters

Parameter	SFT	GRPO (Run 3)
Training Samples	976	2,206
Epochs	3	2
Batch Size (per device)	4	1
Grad. Accumulation	4	4
Learning Rate	5×10^{-5}	2×10^{-5}
Max Sequence Length	—	1024 (intended)
Num. Generations (K)	—	8
Temperature	—	0.8
Loss Type	Cross-Entropy	Policy Gradient

Table A.1: Hyperparameters for SFT and GRPO training.

Appendix B

Error Logs and Debugging Notes

B.1 Critical BF16 NaN Failure

Context: Occurred deterministically at optimizer step 50 with `random.seed(42)`.

```
{'loss': '1.124', 'grad_norm': 'nan', 'epoch': '0.8197'}  
{'loss': '0', 'grad_norm': 'nan', 'epoch': '0.9836'}
```

Resolution: Switched from BF16 to Float32 training.

B.2 Storage Exhaustion

Error:

```
safetensors_rust.SafetensorError:  
  I/O error: No space left on device (os error 28)
```

Resolution: Redirected output directory and HF cache to `/tmp`.

B.3 GRPO Zero-Gradient Episodes

Context: `grad_norm = 0` observed in majority of early GRPO training steps.

Resolution: Added non-binary format reward + increased generation diversity ($K = 8$, `temperature=0.8`).

B.4 Truncation Bug

Context: `max_new_tokens` dropped by `GRPOConfig` with no fallback.

[INFO] `GRPOConfig` dropped unsupported keys: [`'max_new_tokens'`]

Evidence: `completions/clipped_ratio` rose to 0.75–0.95 by end of Run 3.

Resolution: Added `model.generation_config.max_new_tokens` fallback for Run 4.

B.5 Physics Verifier `IndexError`

Error: `IndexError` in `extract_numerical_answer` when “Final Answer:” was followed by empty whitespace.

Resolution: Added guard for empty `splitlines()` result.

B.6 Resolution Summary Table

Component	Error Type	Resolution
SFT	BF16 NaN gradient at step 50	Switched to Float32; reduced LoRA targets to 4 modules, scaling to 1.0
SFT	SFTTrainer API change	Migrated to SFTConfig + ChatML
GRPO	Zero gradient norm	Added format reward + sampling diversity
GRPO	<code>max_new_tokens</code> not applied	Added <code>generation_config</code> fallback
GRPO	Groq API rate limiting	Disabled LLM judge during training
Verifier	<code>IndexError</code> on empty answer	Added empty-list guard
All	<code>OSError 28</code> (disk full)	Redirected to <code>/tmp</code>

Table B.1: Summary of major errors and their resolutions.

Bibliography

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. & Chen, W. (2022), ‘LoRA: Low-rank adaptation of large language models’, *International Conference on Learning Representations* .
- Hu, Z., Xu, Y., Yu, W., Wang, S., Zhao, Z., Lin, X., Tu, Y., Shang, L., Jiang, X. & Liu, Q. (2025), ‘A survey on small language models’, *arXiv preprint arXiv:2501.05465* .
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A. et al. (2022), ‘Training language models to follow instructions with human feedback’, *Advances in Neural Information Processing Systems* **35**, 27730–27744.
- Red Hat AI (2024), ‘Small language models for structured tasks’, <https://techsignal.news/enterprise-ai/red-hat-small-language-models-structured-tasks>. Accessed: April 2026.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017), ‘Proximal policy optimization algorithms’, *arXiv preprint arXiv:1707.06347* .
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y. & Guo, D. (2024), ‘DeepSeekMath: Pushing the limits of mathematical reasoning in open language models’, *arXiv preprint arXiv:2402.03300* .
- Sutton, R. S., McAllester, D., Singh, S. & Mansour, Y. (1999), ‘Policy gradient methods for reinforcement learning with function approximation’, *Advances in Neural Information Processing Systems* **12**.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. & Polosukhin, I. (2017), ‘Attention is all you need’, *Advances in Neural Information Processing Systems* **30**.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H. et al. (2024), ‘Qwen2.5 technical report’, *arXiv preprint arXiv:2412.15115*.